



—
A I - A U G M E N T E D

Paper Submission & Peer-Review System

Reimagining a course project with Generative AI

Bhanu Gupta

Generative AI in Practice · Assessment 1 · May 2026

The original project

A peer-review system I built for a previous web-development course

TECH STACK

- Node.js · Express
- EJS server-rendered views
- SQLite (sqlite3 driver)
- bcrypt + express-session
- multer file uploads
- ExcelJS export

FEATURES

- Author submits manuscript + abstract
- Editor manually assigns reviewers
- Reviewer writes free-text feedback
- Public reader feed of accepted papers

ROLES & FLOW



Limitations of the original

What an honest audit surfaced — and why GenAI is the right cure



Buggy auth wiring

express-session never imported; mixed req.session shape across routes; broken author-id lookup.



Random reviewer assignment

Math.random() picks any reviewer regardless of expertise — a real workshop never operates this way.



No integrity checks

No plagiarism detection, no duplicate-submission detection, no AI-generated-text flagging.



Unstructured reviews

A single textarea per review. No comparable scores, no aggregation, no recommendation field.



No author tooling

Authors get zero feedback before submitting — no abstract polish, no keyword extraction, no title help.



No tests, no Docker, no CI

Single-file app.js, hardcoded session secret, two stray .db files. Won't survive code review.

Why Generative AI?

Each weakness above maps to something today's AI can directly support

4

AI features planned

2

LLM provider backends

0\$

Zero-cost demo mode

5

User roles supported



Generation

LLMs read an abstract and emit structured prose: review drafts, polished text, alternative titles.



Retrieval

Embeddings turn text into vectors — reviewer matching and plagiarism become numeric distance problems.



Decision support

AI never decides. It surfaces signals; humans accept, override, and stay accountable.

Four planned GenAI features

Each addresses a specific weakness from the audit



Author writing assistant

LLM

Polishes the abstract, suggests alternative titles, extracts keywords on demand. Author edits and accepts.



Smart reviewer matching

Embedding
s

Cosine similarity over reviewer expertise tags. Editor sees a top-3 ranked list with scores and can override.



Plagiarism + AI-text flag

Embedding
s

Max similarity to existing corpus + transparent stylometric AI-text score. Flag, never verdict.



AI reviewer assistant

LLM

Drafts structured first-pass review (summary / strengths / weaknesses / 1-5 scores / recommendation).

Architecture I have prepared

Provider-agnostic LLM abstraction, MVC layout, audit logging



Implementation status

What is scaffolded · what is fully wired · what is intentionally pending

Capability	Status	Notes
MVC project structure	DONE	Routes / controllers / services / models / views all in place
Auth, sessions, RBAC for 5 roles	DONE	helmet, CSP, rate-limit, SQLite session store, role middleware
Reviewer matching (TF-IDF + cosine)	DONE	Working today on the heuristic backend — verifiable in the editor dashboard
Plagiarism similarity scoring	DONE	Running on every submission against the existing corpus
AI-text stylometric flag	DONE	Transparent sub-signals exposed to the editor
AI reviewer draft (heuristic templates)	SCAFFOLDED	UI + API in place; switches to real LLM once API key supplied
Author writing assistant (heuristic)	SCAFFOLDED	Polish / titles / keywords scaffolded; LLM call ready to flip on
Claude API integration	PLANNED	Adapter committed; activates with one env var; deferred until budget approved
Sentence-transformer embeddings	PLANNED	Roadmap item for Assessment 2 — replaces TF-IDF for richer matching
User study + benchmarking	PLANNED	Planned for Assessment 2; ai_audit table already collects the data

AI tools I used to build this

How I shipped a 36-file codebase, a report, and this deck in days



Claude

Sonnet 4.6 (chat + code)

Architecture decisions, full code generation, the 1500-word report, this presentation. The main brain throughout.

main coding partner



Whisper Flow

voice → text dictation

Talked through prompts and design ideas instead of typing. Way faster turnaround when iterating on architecture.

prompt at speaking speed



Cursor

AI-native code editor

Inline AI completion and refactoring while editing the EJS views and controller code. Tab-to-accept, repeat.

inline edits

Also in the chain · ChatGPT for second opinions · GitHub Copilot for repetitive code · Excalidraw for diagrams · Pandoc for docx checks

Ethical considerations

Peer review affects careers — we engineer for human accountability



Authorship & accountability

AI drafts; humans submit. ai_assisted flag on every review. The model never autonomously approves anything.



Bias & fairness

LLM bias inherited from training data; expertise tags can under-represent niche fields. Editor override + audit trail mitigate.



Privacy & data minimisation

Defaults to local heuristic backend. When Claude is on, only title + abstract leave the trust boundary. Every external call audited.



AI-text detection limits

Stylometric score is a flag, not a verdict. Sub-signals are exposed; submission is never blocked solely on this score.

Live demo: the recreated v2

Already running — five role-aware dashboards with AI affordances

1

Author submit page

Three AI buttons (Polish · Suggest titles · Extract keywords) sit alongside the abstract.

2

Editor dashboard

Top-3 AI-ranked reviewers per paper with cosine score, plus manual override and final decision.

3

Reviewer review

"Generate AI draft" populates summary / strengths / weaknesses / scores / recommendation.

4

Public reader feed

Newspaper-style grid of accepted articles, full-text search, public download — no login.

TO RUN LOCALLY

```
$ cd paper-submission-system -v2
$ npm install
$ npm run setup
$ npm start
```

→ <http://localhost:3000>

Demo accounts (Password123!):

```
alice / bob — author
reviewer_m l / db / se
editor — manual AI assignment
adm in — audit dashboard
reader — public feed
```

Default mode: heuristic backend (zero cost). Add ANTHROPIC_API_KEY + LLM_PROVIDER=claude to upgrade — no other code changes.

Roadmap to Assessment 2

A five-step plan from today through the Assessment 2 submission



Takeaway: every Assessment-2 experiment has scaffolding waiting for it — the v2 codebase is the platform, not the prototype.



THANK YOU

**Questions, suggestions, or
budget approvals welcome.**

Bhanu Gupta · bhanuguptagarg@gmail.com

Repo: [paper-submission-system-v2](#) · Report: [Assessment_Report.docx](#)